

TECHNICAL DISCUSSION

Meeting Minutes: Understanding Vector Databases

Meeting Topic	Understanding How Vector Databases Work
Attendees	Wen Sun, Yingzhe Xu, Nuo Chen, Xiang Chang, Xingyu Li
Meeting Type	Technical Discussion

Meeting Overview

This meeting focused on the fundamental concepts, operating principles, and practical applications of vector databases. The main topics included the differences between vector databases and traditional databases, how embedding models convert text into vectors, how similarity search works, and how vector databases can be combined with large language models (LLMs) in Retrieval-Augmented Generation (RAG) systems.

1. Vector Databases and Traditional Databases

The team first discussed the main differences between traditional databases and vector databases. Traditional databases are mainly used for exact matching, structured queries, and keyword-based searches. In contrast, vector databases retrieve information according to the degree of similarity between different pieces of content. Vector databases are therefore particularly suitable for semantic search. Two sentences may use completely different words while expressing similar meanings. A traditional keyword search may fail to recognise this relationship, whereas vector-based search can determine whether the two sentences are semantically related. The team agreed that the primary advantage of a vector database is that it can search according to the **meaning** of content rather than relying only on identical keywords.

2. Embeddings and Vector Representations

The team then discussed how vectors are used to represent textual content. An embedding model can convert text, images, or other types of data into high-dimensional numerical vectors. These vectors represent semantic features contained in the original content.

```
"I like cats"
->
Embedding Model
->
[0.13, -0.72, 0.25, 0.81, ..., -0.14]
```

In most cases, individual dimensions within a vector cannot be directly interpreted. What matters is the relative position of different vectors.

Content with similar meanings will normally have vectors that are located close to one another in the embedding space. For example, the vectors corresponding to "I like cats" and "I love kittens" would normally be relatively close, while both would be much farther away from a vector representing a sentence about the stock market.

3. Document Chunking

The team further discussed how long documents are normally processed before being stored in a vector database.

Rather than converting an entire document into a single vector, the document is usually divided into several smaller content segments. This process is called chunking.

```
Document
  ->
Chunking
  ->
Embedding
  ->
Vectors
```

Chunking enables the system to retrieve a specific paragraph or section from a document instead of returning the entire large document every time.

The team also noted that chunk size and segmentation strategy directly affect retrieval performance. If a chunk is too large, it may contain a significant amount of information unrelated to the query. If a chunk is too small, it may lose the contextual information required to understand the content.

4. Information Stored in a Vector Database

The team confirmed that a vector database normally stores more than vectors alone. A typical record may contain:

- An embedding vector
- The original text
- A document ID
- A filename
- A page number
- An author
- A date
- A category
- Other metadata

The vector is mainly used for similarity search, while the corresponding text and metadata help the system identify and use the original information.

Metadata can also be used to filter search results. For example, the system can restrict the retrieval scope according to document type, date, source, or category.

5. User Queries and Similarity Search

The team then discussed what happens inside the system when a user submits a query.

First, the user query is converted into a vector by the same embedding model. This is called a query vector. The vector database then compares the query vector with the vectors already stored in the database and searches for the closest results.

```
User Query
->
Embedding Model
->
Query Vector
->
Vector Search
->
Most Similar Vectors
->
Relevant Content
```

The system will normally use Top-K retrieval to return a specified number of the most relevant results, such as the five or ten most similar chunks.

The team discussed several common similarity calculation methods:

- Cosine similarity
- Euclidean distance
- Dot product

The appropriate method normally depends on the embedding model and vector retrieval system being used.

6. Vector Indexes and Large-Scale Search

The team further discussed the search-efficiency challenges that arise when a database contains millions or even hundreds of millions of vectors.

The most direct method is to compare the query vector with every vector stored in the database. This is known as brute-force search. Although this method can return relatively accurate results, its computational cost becomes extremely high as the size of the dataset increases.

Vector databases therefore normally create specialised vector indexes and use Approximate Nearest Neighbour (ANN) search algorithms.

Some common approaches discussed during the meeting included:

- HNSW
- IVF
- PQ
- IVFPQ
- DiskANN
- ScaNN

The team discussed HNSW in further detail. HNSW can be understood as a graph-based search method. It connects similar vectors and creates a search structure containing multiple layers.

At higher layers, the system can quickly move towards an area that is broadly related to the query. After reaching the lower layers, it performs a more detailed local search.

This approach can significantly reduce the number of vectors that must be compared for each query, thereby improving retrieval efficiency in large-scale databases.

7. Using Vector Databases in RAG Systems

A significant part of the meeting focused on how vector databases can be combined with LLMs. The team discussed the basic architecture of Retrieval-Augmented Generation.

Offline Knowledge-Base Processing

```

Documents
  ->
Parsing
  ->
Chunking
  ->
Embedding
  ->
Vector Database
    
```

Online Query Process

```

User Question
  ->
Embedding
  ->
Vector Database Search
  ->
Relevant Chunks
  ->
LLM
  ->
Final Answer
    
```

In this process, the main responsibility of the vector database is to retrieve relevant information. The LLM is responsible for understanding the retrieved content and combining it with the user's question to generate the final answer.

The team therefore concluded that a vector database can be understood as an LLM's **External Semantic Retrieval System** or **External Memory**.

8. Why Retrieval Is Still Required with Large Context Windows

The team also discussed whether vector databases remain necessary as modern LLMs support increasingly large context windows.

The team concluded that larger context windows can reduce some limitations, but they do not eliminate the need for information retrieval.

For example, an organisation may have a document knowledge base containing several gigabytes of information or more. Supplying the entire knowledge base to an LLM every time a user submits a question would be inefficient and would result in high computational and token costs.

A more reasonable approach is to use vector search to select a small amount of relevant content from the larger knowledge base:

```
Large Knowledge Base
    ->
Vector Search
    ->
5-10 Relevant Chunks
    ->
LLM
```

This approach can reduce:

- Token usage
- Response latency
- Computational costs
- Irrelevant information in the LLM's context

It can also help the model focus more effectively on information that is genuinely relevant to the user's question.

9. Related Technologies Discussed During the Meeting

The team also briefly discussed several commonly used technologies.

Embedding Models

- OpenAI Embeddings
- BGE
- E5

Vector Databases

- Pinecone
- Milvus
- Weaviate
- Qdrant
- Chroma

Vector Index and Search Libraries

- FAISS

The team specifically noted that FAISS is more accurately described as a library for vector similarity search and indexing rather than a complete database.

A complete vector database will normally also provide:

- Persistent storage

- Metadata filtering
- Distributed storage
- Access control
- Data scaling capabilities

Meeting Outcomes

Through this discussion, the team established a shared understanding of the basic vector database workflow:

```
Data
->
Chunking
->
Embedding
->
Vector Storage
->
Vector Index
->
Similarity Search
->
Relevant Information
->
LLM
```

The main conclusions from the meeting were:

1. Vector databases enable semantic search rather than relying only on exact keyword matches.
2. Embedding models convert text and other forms of content into high-dimensional vector representations.
3. Long documents normally need to be divided into chunks before embeddings are generated for each chunk.
4. User queries are also converted into vectors and used to find the most relevant content through similarity search.
5. ANN methods such as HNSW can improve vector search efficiency in large-scale datasets.
6. Vector databases are an important component of many RAG systems.
7. In a RAG architecture, the vector database is responsible for retrieving knowledge, while the LLM generates the final answer based on the retrieved results.

Overall, the team concluded that vector databases provide an efficient method for retrieving semantically relevant information from large knowledge bases. They are particularly suitable for connecting external data sources to LLM-based applications.